Compose Basics and UI Layouts

Student Name: Sachin
Course: Android App Development using Gen AI
Module: Jetpack Compose

Task 1 – Composables Practice Reflection & Planning

Problem 1 – Column Layout Exercise

Problem Summary

This exercise required building a simple UI using multiple Text composables arranged vertically. The layout needed proper spacing and alignment using the Column composable.

Planned Approach (Before Coding)

I planned to use Column as the root composable. Inside it, I would add Text composables for title and description. I would use Modifier.padding(16.dp) for outer spacing and Spacer for vertical gaps between elements.

Reflection After Implementation

I understood how Column arranges elements vertically from top to bottom. Initially, spacing felt slightly confusing, but using Spacer and padding helped create balanced layout. Next time, I would sketch layout before writing code.

Problem 2 – Row Layout Exercise

Problem Summary

This exercise required placing two or more UI elements horizontally side-by-side using Row. Proper alignment and spacing were necessary.

Planned Approach (Before Coding)

I decided to use Row as the parent composable and place Text and Button elements inside it. I would use horizontalArrangement = Arrangement.SpaceBetween and Modifier.padding() to control spacing.

Reflection After Implementation

I learned how Row aligns elements horizontally. Managing alignment and spacing required understanding of modifiers. I realized how important arrangement parameters are for balanced UI.

Planning Checklist Template

1. Carefully read the UI specification
2. Sketch the layout on paper
3. Decide the root composable (Column, Row, or Box)
4. Add child composables
5. Apply modifiers (padding, size, alignment)
6. Preview layout
7. Adjust spacing and alignment
8. Review for accessibility

Final Reflection

Planning before coding improves clarity and reduces mistakes. It helps in selecting correct composables and spacing early. Reflection helps identify weaknesses and improve layout structure. This approach increases efficiency in Compose UI development.

Task 2 – Business Card Layout Design (Pre-Code)

Concept: My Developer Business Card

Purpose

To create a simple and professional business card layout displaying my profile details, role, contact information, and image.

Content Plan

- Name: Sudarshan G
- Role: Android Developer
- Bio: Passionate about AI, Machine Learning, and Full Stack Development.
- Contact: ○ Email ○ LinkedIn
- Image: profile_pic.png

Wireframe Layout (Phone)

Column
→ Image (Top Center, 120dp)
→ Text (Name – 28sp, Bold)
→ Text (Role – 18sp)
→ Text (Bio – 16sp)
→ Row (Icon + Email) →

Row (Icon + LinkedIn)

Spacing:

- 16dp outer padding
- 8dp between text elements
- 12dp between contact rows

Tablet / Landscape Variant

Row
→ Image (Left side)
→ Column (Name, Role, Bio, Contacts on right side)

This improves responsiveness on larger screens.

Resource & Style Plan

Image:

- File Name: profile_pic.png
- Folder: drawable-nodpi
- Optimized size

Text Styles:

- Title: 28sp
- Subtitle: 18sp
- Contact Text: 14sp

Colour Plan:

- Background: White
- Primary Text: Black
- Accent Color: Blue

Accessibility Plan:

- Add contentDescription for image
- Ensure 48dp minimum touch targets
- Maintain sufficient contrast

Reflection

I chose Column as the primary layout for vertical arrangement. Row is used for contact details alignment. Proper spacing and modifiers are important to maintain visual balance. I will test this layout using Preview and emulator before final coding.

Task 3 – Layout Review & Improvement Plan

Review Criteria

Visual Alignment

Elements are centered properly. Spacing is mostly consistent but can be refined.

Readability

Font sizes are readable. Subtitle may need slight increase for better clarity.

Resource Usage

Image must be optimized and not too large. Proper drawable folder usage is important.

Responsiveness

Layout adapts well using Row and Column combination for tablet mode.

Accessibility contentDescription added. Touch targets must remain

minimum 48dp.

Improvement Plan Table

Issue	Proposed Fix	Priority
Subtitle slightly small	Increase to 18sp	High
Image file large	Compress image	Medium Button
spacing tight	Add 12dp spacing	Medium
Contact touch area small	Ensure 48dp height	High

Final Reflection

Evaluating UI layouts helps identify alignment, readability, and accessibility issues before final implementation. Planning and reviewing early reduces rework and improves quality. This process ensures responsive and accessible Compose applications.